



**UNIVERSIDAD NACIONAL EXPERIMENTAL DE GUAYANA
VICERRECTORADO ACADÉMICO
COORDINACIÓN DE INGENIERÍA EN INFORMÁTICA
LENGUAJE Y COMPILADORES (2026-I)**

INFORME DE INVESTIGACIÓN: ESTUDIO COMPARATIVO DE PARADIGMAS, LENGUAJES Y DISEÑO DE DSL

Asignación II: Los Lenguajes de Programación

Autores:

Fernando Centeno (C.I. 30.810.484)
Juan Longart (C.I. 31.882.343)
Adrian Reina (C.I. 31.317.970)
Rafael Rodríguez (C.I. 31.882.367)

Profesor:

Félix Márquez
fmarquez@e.uneg.edu.ve

Ciudad Guayana, mayo de 2026

1. Resumen

El presente informe de investigación tiene como propósito analizar la arquitectura lingüística y computacional que sustenta los lenguajes de programación modernos. Se examina la convergencia multiparadigma (imperativo, orientado a objetos, funcional, lógico y concurrente) y su impacto en el diseño de software. Para contrastar diferentes filosofías de diseño, se realiza un estudio morfológico y sintáctico a bajo nivel de cuatro lenguajes representativos: Zig, Rust, Python y JavaScript. Adicionalmente, se evalúa empíricamente el rendimiento de los compiladores e intérpretes frente a procesamiento intensivo mediante un benchmarking basado en el algoritmo de la Conjetura de Collatz para calcular la longitud de secuencia hasta $N=2.000.000$. Los resultados de este análisis empírico evidencian la superioridad en eficiencia de los lenguajes de compilación nativa, como Rust y Zig, destacando sus tiempos de ejecución en milisegundos y la gestión determinista de memoria frente a las penalizaciones sufridas por la máquina virtual de Python. Finalmente, para aplicar la teoría de diseño de lenguajes, se presenta la especificación formal de un Lenguaje de Dominio Específico (DSL) denominado Lenguaje L. Este lenguaje está arquitectado para el control lógico y seguro de un sistema de microredes eléctricas inteligentes y almacenamiento de energía (ECO-GRID), demostrando cómo las abstracciones sintácticas delimitan el espacio de errores en entornos industriales. En conclusión, el estudio confirma que las decisiones de diseño a nivel léxico y sintáctico determinan de manera fundamental el desempeño computacional, la expresividad y la viabilidad operativa de las soluciones tecnológicas.

2. Introducción

En el desarrollo de software y en las ciencias de la computación, el estudiante comúnmente interactúa con los lenguajes de programación como un usuario final o programador aplicativo. En este nivel inicial, el enfoque principal radica en traducir lógica pura a una sintaxis particular con el fin de resolver problemas de complejidad moderada. Sin embargo, al alcanzar el nivel del diseño de compiladores e intérpretes, se vuelve indispensable contemplar el lenguaje no como una herramienta inalterable, sino como un producto de software minuciosamente diseñado, provisto de una arquitectura lingüística específica y sujeto a restricciones formales rigurosas.

Las decisiones de diseño léxico, morfológico y sintáctico tomadas por los diseñadores de lenguajes impactan directamente la eficiencia de la ejecución, la expresividad del código, el consumo de memoria y la mantenibilidad a largo plazo.

Bajo esta premisa, el presente documento profundiza en la evolución y convergencia de las filosofías de diseño de software contemporáneas. Se busca ir más allá de la codificación tradicional para comprender, desde las bases de la infraestructura computacional, cómo la gestión interna de memoria y el análisis sintáctico diferencian a lenguajes orientados a sistemas frente a entornos dinámicos o interpretados. Asimismo, se plantea el desafío práctico de aplicar esta teoría morfológica para abstraer un entorno físico real en una gramática computacional propia, demostrando el valor de los lenguajes diseñados a medida para entornos industriales críticos.

3. Actividad I: Matriz Descriptiva y Análisis de Paradigmas

La convergencia tecnológica moderna ha impulsado a la industria hacia entornos multiparadigma. Lenguajes originados bajo filosofías rígidas ahora asimilan propiedades de modelos ajenos para mitigar sus deficiencias nativas. Comprender esta transición requiere una inmersión en los fundamentos matemáticos y operativos de cada paradigma.

3.1. El Paradigma Imperativo y la Gestión del Estado Estructural

El paradigma imperativo representa la manifestación de software más fiel a la arquitectura física de Von Neumann. Su ontología se fundamenta en la manipulación explícita del estado del sistema computacional mediante sentencias secuenciales. En este modelo, las variables operan como representaciones directas de celdas físicas de memoria que sufren alteraciones indiscriminadas a lo largo del ciclo de vida del programa. La base operativa es la mutabilidad, una filosofía de diseño que privilegia algoritmos de alta velocidad y un consumo de memoria reducido, dado que las estructuras de datos complejas se sobrescriben directamente sobre la memoria RAM (Random Access Memory) en lugar de requerir costosas reasignaciones estructurales.

No obstante, esta dependencia de la mutabilidad introduce vulnerabilidades analíticas severas, destacando el fenómeno de los efectos secundarios. En un entorno puramente imperativo, la invocación de una subrutina puede alterar un estado global o parámetros pasados por referencia de formas no documentadas explícitamente en la firma de la función. Desde la perspectiva del diseño de un compilador, los efectos secundarios complican drásticamente la creación de Árboles de Sintaxis Abstracta (AST) optimizados. Los ingenieros de compiladores deben implementar heurísticas pesadas, como el análisis de alias, para rastrear si múltiples punteros apuntan a la misma región de memoria. Si el compilador no puede garantizar que una variable permanezca inalterada durante una secuencia de instrucciones compartidas, se ve obligado a deshabilitar optimizaciones agresivas de reordenamiento de caché o paralelización automática. Asimismo, el programador debe gestionar manualmente el control de flujo mediante sentencias de bucle e iteración, asumiendo la responsabilidad directa sobre la microgestión temporal del procesador.

3.2. Evolución hacia la Programación Orientada a Objetos (POO)

Para solventar la entropía del estado global en bases de código masivas, la Programación Orientada a Objetos abstrae los datos y los comportamientos en entidades lógicas discretas denominadas objetos. La innovación arquitectónica central del paradigma orientado a objetos es el encapsulamiento, el cual restringe el acceso directo a la representación topológica de los datos internos, exponiendo únicamente interfaces controladas y predecibles.¹ Este aislamiento semántico protege la integridad del estado del programa frente a modificaciones externas, permitiendo que subsistemas complejos sean desarrollados independientemente sin colisiones en el espacio de nombres.

Un pilar del paradigma orientado a objetos es el polimorfismo, que es la capacidad matemática y operativa de una referencia abstracta para manifestar múltiples formas subyacentes. La implementación de esta propiedad exige que los compiladores introduzcan mecanismos de enlace tardío (late binding). Generalmente, esto se logra mediante la construcción de tablas de métodos virtuales (v-tables) almacenadas en memoria. Cuando un programa invoca un método polimórfico, el procesador debe realizar una búsqueda indirecta a través del puntero de la v-table de la instancia para localizar la dirección de memoria exacta de la rutina que debe ejecutarse. Esta indirección introduce una sobrecarga de latencia que, aunque marginal, se vuelve un factor limitante en sistemas de tiempo real.

Históricamente, los modelos orientados a objetos priorizaron la herencia jerárquica para modelar relaciones del tipo "es-un", estructurando taxonomías profundas que compartían código desde clases base hacia clases derivadas. Sin embargo, la acumulación de décadas de ingeniería de software evidenció el problema de la "clase base frágil", donde una alteración en la raíz de la jerarquía provocaba efectos colaterales en cascada, fracturando sistemas distantes. En consecuencia, el diseño contemporáneo ha pivotado agresivamente hacia la composición. Al priorizar relaciones del tipo "tiene-un", la composición ensambla sistemas complejos conectando componentes independientes a través de interfaces, una filosofía que maximiza la reutilización y es nativamente impulsada en lenguajes modernos de sistemas a través de mecanismos como los rasgos (traits).

3.3. Paradigma Funcional y el Sistema Formal del Cálculo Lambda

A diferencia de los enfoques vinculados a la maquinaria física, el paradigma funcional ignora la noción de transición de estado. Su base teórica es el cálculo lambda, un marco matemático y lógico desarrollado por Alonzo Church en la década de 1930 para investigar la computabilidad y los fundamentos de las matemáticas. El cálculo lambda establece un lenguaje universal minimalista donde todo cómputo se expresa utilizando únicamente abstracciones y aplicaciones de funciones continuas, demostrando ser equivalente en poder a una máquina de Turing.

La sintaxis del cálculo lambda opera bajo tres reglas de transformación intrínsecas que los compiladores funcionales contemporáneos implementan directamente en sus fases de optimización. La primera es la conversión alfa, que permite el renombre seguro e indoloro de variables ligadas dentro de una abstracción para evitar colisiones léxicas. La segunda es la reducción beta, que constituye el motor real del cómputo; representa el proceso matemático de aplicar una función sustituyendo cada aparición del parámetro en el cuerpo de la abstracción con el argumento proporcionado. Finalmente, la conversión eta formaliza el principio de extensionalidad, dictaminando que dos funciones son absolutamente equivalentes si producen idénticos resultados para cualquier posible conjunto de entradas.

La asimilación de estos fundamentos en la programación genera arquitecturas basadas en la inmutabilidad de datos y en tratar a las funciones como ciudadanos de primer orden. En este régimen, los valores no pueden mutar una vez instanciados; las operaciones originan copias transformadas. La eliminación estricta de efectos secundarios engendra la propiedad de transparencia referencial, un axioma que estipula que una función evaluada con parámetros concretos siempre devolverá el mismo resultado sin interactuar con recursos externos del hardware. La transparencia referencial provee a los compiladores con garantías absolutas que facilitan tácticas como la memorización, donde la respuesta de rutinas computacionalmente costosas se almacena en caché para evadir recálculos. Asimismo, la ausencia de mutaciones sobre variables compartidas erradica las condiciones de carrera, permitiendo que la distribución multihilo de los cálculos sea matemáticamente trivial y nativamente

segura. Adicionalmente, el paradigma funcional introduce la evaluación perezosa, postergando la ejecución de una rutina hasta que su valor terminal sea imperativamente requerido. Esta mecánica difiere las asignaciones en el heap de memoria, viabilizando algoritmos capaces de procesar estructuras topológicas y listas teóricamente infinitas.

3.4. Paradigma Lógico Declarativo y Deducción Automática

El paradigma declarativo desliga al diseñador de software del control secuencial de la máquina, concentrando el esfuerzo cognitivo en especificar axiomáticamente "qué" reglas rigen el dominio del problema en lugar de dictaminar "cómo" la Unidad Central de Procesamiento debe calcularlas. La cúspide teórica de este modelo se encuentra en la programación lógica, cuyo esqueleto formal reposa sobre las cláusulas de Horn y el álgebra relacional.

El corazón mecánico subyacente en la programación lógica clásica, como el lenguaje Prolog, es el algoritmo de unificación, descubierto y axiomatizado por Alan Robinson en 1965. Este hito en la deducción matemática instruye a la máquina sobre cómo emparejar dinámicamente dos expresiones lógicas para encontrar su Unificador Más General. El algoritmo iterativamente aplica sustituciones sobre las variables libres hasta lograr que ambas ramas semánticas sean estructuralmente idénticas. La belleza matemática del hallazgo de Robinson radicó en garantizar la resolución automática de teoremas evitando ramificaciones superfluas que agotarían exponencialmente los recursos del sistema.

En la praxis del diseño de compiladores y motores lógicos modernos, existen bifurcaciones vitales en la evaluación de estas reglas. Prolog utiliza un enfoque de evaluación top-down (descendente) sustentado en retroceso automático (backtracking). Cuando una resolución fracasa, el intérprete retrocede en el árbol de dependencias probando la siguiente sustitución posible hasta satisfacer la meta. Paralelamente, extensiones modernas enfocadas en seguridad y bases de datos deductivas, como Datalog, favorecen una ejecución bottom-up (ascendente). Datalog prohíbe sintácticamente las funciones compuestas recursivas dentro de sus cláusulas para forzar una terminación polinomial determinada. Este aislamiento de la complejidad infinita ha catapultado a Datalog como la base para el análisis estático de código fuente, empleándose internamente en infraestructuras avanzadas como Soufflé para mapear árboles

de sintaxis abstracta masivos y ensamblar sistemas de validación de tipos sin las ambigüedades derivadas de rutinas recursivas profundas.

3.5. Paradigma Concurrente: Modelos de Actores y Procesos Secuenciales

A medida que las barreras físicas en el escalamiento de la frecuencia de reloj de la CPU estancaron los avances térmicos, el diseño de lenguajes integró abstracciones algorítmicas de paralelismo. Durante la década de 1970, nacieron dos teorías matemáticas rivales pero complementarias para lidiar con el infierno del paralelismo basado en cerraduras de memoria: el Modelo de Actores promulgado por Carl Hewitt, y los Procesos Secuenciales Comunicantes (CSP) descritos por Tony Hoare.

El modelo CSP, incorporado magistralmente en entornos como Go (mediante goroutines), fundamenta su concurrencia en la transferencia temporal de la propiedad a través de intermediarios formales denominados "canales". En CSP, los subprocesos que ejecutan código son esencialmente anónimos y carecen de identidades directas, centrando el diseño tipográfico del compilador alrededor del canal. Frecuentemente, las comunicaciones CSP imponen un comportamiento sincrónico o de punto de encuentro (rendezvous), estipulando que el proceso emisor debe bloquearse suspendiendo sus ciclos de procesador hasta que el receptor esté posicionado para drenar el canal, garantizando un acoplamiento temporal muy estrecho que previene el desbordamiento de colas y fallos de falta de memoria en sistemas altamente saturados.

En abierta divergencia topológica, el Modelo de Actores concibe un universo computacional donde cada nodo de procesamiento (el Actor) ostenta una identidad directa, privada y enrutable (PIDs en máquinas virtuales como la de Erlang). En contraposición al modelo de Hoare, los actores se comunican por medio del paso asíncrono de mensajes canalizados a través de buzones incrustados dentro del propio actor receptor. Carl Hewitt diseñó esta infraestructura para evadir por completo el concepto de bloqueo. Un actor procesa estímulos de su buzón en aislamiento estricto, capaz de mutar su propio estado encapsulado, instanciar actores secundarios de forma masiva o inyectar nuevas respuestas a la red, todo ello sin ceder el control en la espera de confirmaciones de recepción. Al prohibirse absolutamente la memoria mutable de uso compartido a nivel de la máquina virtual, se erradican los bloqueos mutuos

(deadlocks) y se democratiza la distribución de procesos; para el sistema y el lenguaje, resulta sintáctica y operativamente transparente si el mensaje se despacha hacia el microprocesador local contiguo o hacia un clúster físico albergado en un continente distinto.

3.6. tabla

Paradigma	Unidad Estructural	Manejo de Estado	Base Matemática / Control de Flujo	Orientación Central
Imperativo	Sentencia y Variable	Mutable Compartido	Arquitectura Von Neumann (Secuencial)	"Cómo" procesar el estado en la máquina
Orientación a Objetos	Objetos e Interfaces	Encapsulado en clases	Polimorfismo, Despacho Dinámico, Enlace Tardío	Abstracción de datos jerárquica y composición
Funcional	Función Pura	Inmutable y Efímero	Cálculo Lambda de Church (Reducciones Alfa, Beta, Eta)	"Qué" calcular sin interacción con recursos externos
Lógico	Hechos y Reglas	Transparente / Declarativo	Algoritmo de Unificación de Robinson y Cláusulas de Horn	Búsqueda heurística y resolución de predicados
Concurrente	Actores o Canales	Aislado o Transferido	Modelos matemáticos de Hewitt y Hoare (CSP)	Intercambio de mensajes asíncronos o bloqueantes

Tabla 1. Síntesis de Paradigmas de Programación

4. Actividad II: Estudio Comparativo de Lenguajes y Benchmarking

Para observar cómo los axiomas abstractos de los paradigmas teóricos descritos previamente convergen en aplicaciones arquitectónicas del mundo real, se disecciona la anatomía interna de los entornos tecnológicos de Zig, Python, Rust y JavaScript. El estudio se desplaza deliberadamente desde el análisis superficial del desarrollador aplicativo para internarse en las estructuras de generación de código, asignación de memoria a nivel de bits y rutinas gramaticales, obviando el benchmarking estadístico.

4.1. Zig Introspección Semántica, ZIR y Ausencia de Ocultamiento de Memoria

Zig se perfila como la evolución inminente del paradigma imperativo de desarrollo de sistemas y kernels, sustituyendo el comportamiento oculto heredado de los compiladores de C tradicionales por un modelo fundamentado en las representaciones intermedias explícitas y metaprogramación unificada.

Tubería de Compilación y la Representación ZIG

El ciclo de vida del proceso de compilación en Zig despliega una tubería meticulosamente estructurada con un imperativo principal: mantener un consumo minúsculo de memoria RAM. El flujo se inicia con la tokenización léxica y el análisis sintáctico (parsing) produciendo un Árbol de Sintaxis Abstracta (AST) jerárquico. Sin embargo, la persistencia prolongada de árboles masivos en memoria debilita a muchos compiladores modernos. Para evadir esto, la tubería invoca un módulo central designado AstGen. Este subsistema transforma directamente los nodos efímeros del AST en una estructura compacta denominada Representación Intermedia de Zig (ZIR).

El formato ZIR es un código intermedio de instrucciones no tipadas diseñado de forma global por cada archivo fuente. Su proeza de optimización subyace en la "internación de cadenas". Los identificadores, comentarios, y cadenas literales nunca se duplican en la memoria de las estructuras sintácticas; son condensados en un arreglo singular continuo de bytes del sistema. Las instrucciones en ZIR simplemente almacenan diminutos índices numéricos

apuntando hacia estos desplazamientos en el arreglo maestro, colapsando dramáticamente el consumo de memoria del proceso de compilación. Una vez emitidas todas las instrucciones ZIR, el analizador destruye y libera todo el árbol AST original y la memoria del código fuente textual original, procediendo con la compilación sin depender de su huella de arranque.

En la etapa adyacente, el módulo de Análisis Semántico (Sema) adopta un comportamiento poco ortodoxo al funcionar internamente como una máquina virtual que actúa e interpreta de manera recursiva sobre las directivas ZIR. Sema efectúa las comprobaciones de tipos rigurosas y resuelve las dependencias, mutando la estructura genérica ZIR hacia la Representación Intermedia Analizada (AIR), que es un dialecto simplificado, lineal y fuertemente tipado que luego será entregado a los módulos generadores del backend, como la infraestructura genérica MIR de LLVM o los ensambladores directos.

Evaluación Comptime y Transparencia Referencial Cruzada

A diferencia de lenguajes antiguos que inyectan metaprogramación a través de preprocesadores de macros de manipulación de cadenas textuales, Zig integra todo el poder generativo de macros utilizando la misma sintaxis pura de Zig a través del contexto comptime. Cuando el módulo intérprete Sema cruza una directriz marcada con la palabra reservada comptime, frena la emisión de instrucciones hacia el formato AIR y procesa la directriz matemáticamente dentro del propio compilador.

Esta infraestructura obedece a un modelo de transparencia referencial absoluto sin inyectar evaluaciones ocultas del anfitrión. En escenarios de compilación cruzada, si el programador expide una directiva comptime para conocer el límite arquitectónico nativo de un entero de 32 bits, el bloque de evaluación de Sema no leerá el tamaño de registro del hardware local del programador (host), sino que simulará y evaluará las restricciones de la arquitectura remota dictaminada en las variables de entorno, impidiendo fugas de estado algorítmicas que usualmente envenenan los binarios compilados en entornos multiplataforma.

Arquitectura Estática de Allocators y Garantía de Defer

La directriz filosófica cardinal en Zig rechaza rotundamente la presencia de cualquier control global de memoria o recolección de basura asíncrona. Las interfaces del sistema exigen que cualquier función responsable de invocar memoria dinámica reciba formalmente en sus argumentos una estructura que implemente la lógica de asignación, como `std.mem.Allocator`. Este pragmatismo explícito propulsa la adopción en programación embebida e ingeniería aeroespacial; el compilador no forzará la invocación de llamadas al sistema operativo si el desarrollador decide suministrar un `FixedBufferAllocator`, que secuestra y opera en rebanadas estáticas puras alocadas en el stack (pila temporal) o mediante estructuras tipo "Arena".

Por otro lado, la liberación manual directa incurre clásicamente en vulnerabilidades letales en flujos de control anidados complejos (fugas de memoria o "use-after-free"). El analizador sintáctico mitiga estas falencias inyectando primitivas de control conocidas como `defer` y `errdefer`. Cuando se lee un `defer`, el generador semántico compila y guarda rutinas terminales en una pila del árbol de código. Sin importar cuántas bifurcaciones de error devuelva la función prematuramente, el analizador AIR impone la ejecución de desasignación incondicional garantizada, replicando matemáticamente las mecánicas de seguridad sin la entropía de recolectores de basura ocultos.

4.2. Python Metamorfosis Sintáctica y Tolerancia Generacional en Ciclos

Como referente incuestionable del ecosistema multiparadigma científico, CPython (la principal implementación del lenguaje) encarna un modelo arquitectónico que compensa su asimetría en rendimiento con flexibilidades lexicológicas extremas y tolerancia estructural.

Topología Lexicológica Off-Side y la Reingeniería PEG

El diseño léxico original del entorno recae en el sistema algorítmico geométrico ideado por Peter J. Landin en 1966, apodado "regla off-side". Mientras lenguajes derivados de la herencia de C delimitan ámbitos léxicos y bucles con corchetes fijos, Python asume la profundidad de indentación estructural para agrupar operadores semánticos. Esta simplificación tipográfica

impone una sobrecarga computacional sustancial a los analizadores de código. Un autómata de estado finito tradicional falla al lexear Python, requiriendo que la primera etapa del flujo mantenga dinámicamente una pila de registros de espacios en blanco por cada retorno de carro detectado. Si el conteo de espacios en una línea sucesiva sobrepasa a la cúspide de la pila, la máquina inserta a la fuerza el token sintético INDENT. Al detectar un decremento abrupto, el motor entra en modo destructivo expulsando recursivamente registros de la pila generando cascadas de operaciones DEDENT hasta restaurar la equidad topológica del margen correspondiente.

Posterior al léxico, la infraestructura sintáctica de Python atravesó una de las reconstrucciones más ambiciosas en su historia contemporánea. Hasta iteraciones relativamente maduras del lenguaje, el compilador operaba bajo un analizador derivado de la familia generadora LL(1). Este algoritmo LL(1) dependía de la inspección predictiva unida de mirar un solo token (lookahead de un token) para dirimir hacia qué bifurcación gramatical transitar. Frecuentemente esta restricción permitía la inclusión temporal de sintaxis anómala e ilegal, obligando al modelo a crear un Árbol de Sintaxis Concreta (CST) intermedio excesivo solo para rechazar estructuras improcedentes fases más adelante. La modernización formalizada bajo el Documento de Mejora PEP 617 descartó el parser anticuado y lo sustituyó de forma definitiva por un Analizador Basado en Expresiones Gramaticales (PEG). La nueva arquitectura de máquina de análisis PEG ignora el árbol CST generando estructuras de nodos de sintaxis abstracta directamente. Para eludir los bucles de bloqueo infinitos que ocurren convencionalmente durante escenarios de "recursión por la izquierda" en gramáticas complejas, el nuevo módulo integró sistemas de memorización en caché estáticos, otorgando mayor ductilidad a los diseñadores lingüísticos de Python para inyectar sentencias condicionales estructuralmente complejas (como pattern matching avanzado) impensables bajo su motor original.

Fugas Circulares y Análisis en la Máquina Virtual

Durante su fase en tiempo de ejecución, CPython traduce el código abstracto a formato Bytecode y procede a procesarlo en su Máquina Virtual delegada, que implementa estructuras de datos en C (objetos denominados genéricamente PyObject) con pesados metadatos para la inferencia dinámica.¹ La disciplina operativa de gestión de la memoria RAM principal subyacente es el

Conteo de Referencias (Reference Counting). Bajo esta disciplina temporal y síncrona, cada objeto incrementa y decrementa una cuenta atómica a medida que interactúa con el flujo del programa. Tan pronto la variable local pierde referencia, la cuenta se desploma a cero liberando el objeto.

Sin embargo, el axioma matemático del Conteo de Referencias colapsa letalmente ante bucles de auto-referencia y referencias circulares encadenadas, tales como árboles bidireccionales, o una anomalía común donde una interrupción de excepciones guarda información del rastro de la pila (traceback) que indirectamente almacena instancias variables locales que apuntan al registro de la propia excepción.⁴⁵ Para salvaguardar al sistema operativo, CPython apoya su arquitectura insertando el módulo periódico gc. Este co-procesador interno aísla las entidades de objetos contenedores utilizando listas de datos doblemente enlazadas, inyectando un rastreo heurístico y recursivo. Todas las estructuras deben cumplir la directriz algorítmica `Py_TPFLAGS_HAVE_GC` para declarar soporte. El gc atraviesa internamente los árboles utilizando interfaces embebidas C API denominadas `tp_traverse` permitiendo identificar el ciclo circular aislado de dependencias. Una vez identificado, se activa un protocolo de choque en el objeto ofensor mediante la interfaz C `tp_clear`, que rompe brutalmente el bucle expulsando los conteos internos hasta devolverlos artificialmente a cero.

4.3. Rust Tuberías Incrementales y Deducción Relacional de Préstamos

La estructura profunda del lenguaje de compilación anticipada (AOT) Rust, es el cruce indiscutible entre determinismo y abstracciones funcionales sin compromisos con rastreadores de memoria en ejecución.

Modelado de Compilación por Consultas (Salsa)

Para administrar tiempos de transmutación y enlazado excesivos en proyectos monolíticos a gran escala, Rust y su infraestructura de análisis paralela (`rust-analyzer`) transformaron drásticamente el modelo algorítmico lineal de compilación histórica implementando redes granulares basadas en "Consultas" o query-driven systems (cuyo epítome de estudio se encapsula en la biblioteca central Salsa y su derivación nativa de `rustc`).

En la arquitectura de Salsa, todo el ciclo de transmutación del proyecto no procesa etapas monolíticas, sino que evalúa el estado bajo una red masiva de

funciones lógicas puras. Identifica los archivos del sistema operativo como los únicos "inputs base", los cuales alimentan redes dependientes de transformaciones que extraen "valores derivados" abstractos (como el mapeo de los macroprocesadores o el tipado estructural inferido). Esta maquinaria aprovecha el algoritmo rojo-verde (red-green algorithm) que almacena masivamente datos generados por memoización. Si el usuario interrumpe una instrucción diminuta incrustada dentro de una clase inferior, el ecosistema no desecha los resultados del tipado del proyecto. Rastrea el nodo de dependencia topológica impactado invalidándolo recursivamente por debajo, forzando actualizaciones finamente detalladas que evitan que subsistemas como la Infraestructura Generadora LLVM y las ramificaciones del backend inviertan latencia recalculando abstracciones ya resueltas exitosamente.

Comprobadores Matemáticos, NLL y Orígenes Deductivos (Polonius)

La arquitectura de gestión de memoria de Rust rechaza explícitamente a los recolectores de basura, operando su control en la frontera de traducción mediante el Comprobador de Préstamos (Borrow Checker). Las encarnaciones iniciales de Rust implementaban "Tiempos de Vida Léxicos" estrictos, deduciendo que la longevidad útil de un préstamo terminaba obligatoriamente en el corchete literal final del bloque de alcance `}`. Esta severidad artificial de alcance generaba frustración, dado que el compilador no observaba el flujo operativo intermedio. Para solventarlo, Rust iteró la arquitectura de su Comprobador basándolo en su Nivel de Representación MIR, inyectando Tiempos de Vida No Léxicos (NLL) que efectúan un escaneo liveness analysis, detectando las caídas biyectivas de variables según las bifurcaciones lógicas dentro de las secuencias de control del Grafo de Flujo (CFG).

Pese al modelo NLL, la inferencia heurística inter-funcional sigue colapsando bajo ciertas topologías que devuelven referencias mutuamente excluyentes (esquemas como `unwrap` de un condicional temprano de tabla hash) obligando a reconstrucciones dolorosas al ingeniero.⁶¹ En respuesta a esto, el sistema se halla desarrollando su arquitectura formal definitiva bajo el marco deductivo apodado "Polonius". El esqueleto conceptual de Polonius no ve a los préstamos de las referencias mutables e inmutables como entidades efímeras estáticas temporales, sino como ecuaciones de álgebra relacional derivadas de la base formal del sistema Datalog.

Polonius modela la topología de la memoria a través de una base de datos

de hechos, como `loan_issued_at` y jerarquías relacionales de subconjuntos mediante `known_placeholder_subset` operando bajo un entorno regido por el concepto formal de "orígenes".⁶⁵ El verificador de Rust genera ecuaciones lógicas intersecando las reglas y la ruta del árbol MIR; un error ilegal (o condición de carrera interceptada) ocurre matemáticamente si, y sólo si, un subconjunto de préstamos genera un predicado deductivo contradictorio que infringe la axiomatización de permisos superpuestos en la inferencia relacional del momento de tiempo. Esta arquitectura Datalog pura cimentará por fin la evaluación analítica inyectable que garantice algoritmos de referencias circulares a escala.

4.4. JavaScript (Motor V8): Inducciones Léxicas ASI y Arquitectura JIT en Fases

Nacido empíricamente para ejecutar rutinas asíncronas ligeras en navegadores remotos, ECMAScript (JavaScript) y su maquinaria líder de ejecución (Google V8), operan sobre paradigmas de transmutación de comportamiento constante que priorizan el inicio veloz y los subprocesos concurrentes.

La Trampa Lexicológica de la Inserción Automática (ASI)

El analizador de la gramática de ECMAScript implementa una flexibilización morfológica radical al eliminar la obligatoriedad matemática de delimitar instrucciones secuenciales con un carácter estricto de punto y coma `;`. Para no desmoronarse internamente durante el parseo de tokens abstractos, el compilador debe desplegar una heurística especulativa de resolución de errores y clausura semántica designada Inserción Automática de Puntos y Comas (ASI).

Esta rutina algorítmica aplica reglas predictivas si el flujo direccional se interrumpe.⁷¹ Al encontrar un "token ofensivo", inoperable con la regla gramatical preexistente anterior, pero mitigado por la separación de espacios y terminadores de línea de retorno de carro, ASI intercepta el sistema insertando silenciosamente la clausura del punto y coma y reiniciando el analizador. A pesar de la usabilidad teórica, ASI provoca fallos severos de control de flujo en códigos imperativos. Uno de los riesgos analíticos más agresivos delinea la

clausura anómala sobre las palabras reservadas operacionales `return`, `throw`, o `yield`. Si un ingeniero indenta un objeto a retornar colocándolo limpiamente en el bloque siguiente, el algoritmo ASI aplicará forzosamente la regla de finalización antes del objeto. Como resultado, la máquina en tiempo de ejecución abandonará el bloque retornando variables genéricas `undefined` vacías ignorando irreversiblemente las declaraciones en las capas sintácticas subyacentes, creando código muerto inoperante.

Máquina Virtual Just In Time (JIT) y Clases Ocultas

A diferencia de CPython y Rust, el modelo V8 esquivo la interpretación cíclica tradicional empotrándose en una tubería enlazada de compilación Just In Time (JIT), desplegada en etapas escalonadas asimétricas.

1. La primera fase JIT recae en Ignition, un intérprete altamente sensible capaz de ingerir y procesar código de bytes puro para maximizar el tiempo de respuesta y acelerar bucles iniciales crudos.
2. A medida que ciertos fragmentos lógicos elevan su consumo iterativo, la arquitectura canaliza el bytecode de Ignition pasándolo incondicionalmente a la capa de Sparkplug. Como compilador de ráfaga rápida, Sparkplug emite bloques de instrucciones de máquina física linealmente omitiendo evaluaciones profundas o matemáticas abstractas.
3. Simultáneamente, el motor evalúa heurísticamente si el segmento califica como una ruta caliente (hot path). En fases de carga alta de cómputo, delega el código al optimizador intermedio recién concebido, Maglev, o directamente al colosal optimizador TurboFan. TurboFan transforma los códigos y secuencias semánticas integrándolas a gráficos continuos del Árbol Sintáctico (conocido en los marcos de ingeniería como enfoques "Sea of Nodes"), despiezando de manera drástica las dependencias funcionales para reensamblarlas directamente en un código fuente ensamblador ultrarrápido.

El fundamento de este ensamblaje es la recolección previa de estados a través de "Clases Ocultas" (Hidden Classes o Maps). Dado que ECMAScript expone la creación polimórfica incontrolable de propiedades dinámicas sin la jerarquía de estructuras estáticas impuestas, TurboFan no podría compilar eficientemente. Para contrarrestar esta vulnerabilidad, el sistema V8 intercepta dinámicamente cada iteración estructural; siempre que dos diccionarios JSON

dis pares exponen las mismas rutas, crea una clase virtual transitoria apuntalada en vectores de memoria conocidos como DescriptorArrays. Mediante esta clase transitoria forzada, se logran desplegar Cachés en Línea (ICs - Inline Caches). Si un comportamiento repite accesos de memoria bajo esquemas inamovibles, la tubería lo evalúa como un estado "Monomórfico", permitiendo un acceso brutal a velocidades C.83 Cualquier interrupción con polimorfismo agresivo descarrila las asunciones del IC provocando una dolorosa "Des-Optimización", donde V8 colapsa la estructura optimizada abortando en los esquemas rudimentarios de la red de bajo nivel del intérprete Ignition.

Pipeline Orinoco de Gestión Generacional de Memoria

Toda la transmutación de JIT requiere vastas infraestructuras para soportar variables alojadas asíncronamente; una disciplina solventada por V8 bajo el mecanismo algorítmico Orinoco, que adopta explícitamente estrategias de aislamiento generacional.

La partición generacional escinde a las capas del heap global para maximizar el procesamiento. El Espacio Joven, reservado para variables temporales y rápidas asíncronas, emplea un modelo de recolección de eventos apodado "Scavenger" bajo iteraciones derivadas estrictas del algoritmo formal de Cheney.⁸⁹ Scavenger se fundamenta en mitades topológicas conocidas como sub-espacios enlazados From-Space y To-Space.⁸⁸ Todos los apuntadores residen en From-Space, lo que acelera el incremento constante de punteros. Al desbordarse físicamente el margen, en lugar de barrer datos basura cíclicamente, el recolector menor localiza heurísticamente el minúsculo reducto de procesos de raíz que continúan interactuando en la cadena viva; el recolector extrae estos elementos copiándolos a extrema velocidad hacia el limpio y desfragmentado To-Space. Posteriormente, la máquina colapsa el sub-espacio From original mediante simple reescritura atómica, invirtiendo la carga magnética e implementando limpiezas en red asimétricas sumamente rápidas que ignoran variables muertas.

Finalmente, los nodos y datos que perviven tras repetidas depuraciones de iteración menor Scavenger experimentan promociones hacia el Espacio Viejo en V8.⁸⁸ Puesto que el arrastre masivo de bloques profundos arruinaría las memorias, este espacio despliega un marco híbrido designado Mark-Sweep-Compact y marcaje concurrente (Concurrent Marking).⁸⁸ La filosofía orquesta barridos recursivos asíncronos distribuyendo el marcado y trazo de vida en sub-

hilos (helper worker threads) transparentes que fluyen simultáneamente y en paralelo mientras el hilo principal (Main Thread) mantiene la lectura secuencial de peticiones del ciclo de eventos sin paralizar operaciones de I/O de gran densidad, finalizando con un barrido (Sweep) y reubicando celdas disgregadas (Compacting) tras alcanzar los techos dinámicos del sistema sin incurrir severamente en paradas absolutas de sistema.

4.5. TABLA

Especificación	Lexical y Semántica (Parsing)	Modelo Funcional de Ejecución	Disciplina de Asignación de Memoria
Zig	AstGen destructivo a formato genérico interno ZIR	Tubería Analítica (Sema y AIR). Transparencia y simulación matemática para macros en directrices comptime	Manipulación Categórica Manual explícita a través de Allocators modulares. Destrucciones estáticas con inferencia defer/errdefer. Ausencia de GC
Python	Tolerancia dinámica tipo Off-Side e iteración basada en Pilas. Generadores directos sintácticos PEG (CST eludidos)	Entorno interpretado CPython con jerarquización VM y variables unificadas dependientes del contexto léxico	Recolector híbrido; Conteo de Referencia lineal acoplado intermitentemente al explorador cíclico C API para destrucción de anomalías circulares encadenadas
Rust	Topología Incremental Basada en Datos (Redes de Consultas Salsa/rustc) almacenando metadatos para actualizaciones	Despliegue anticipado puro (AOT) derivado del modelado estructurado LLVM nativo	Extracción Deductiva Polonius; base matemática de Relaciones y Orígenes en cruces de nodos CFG validando vida de punteros estáticos

JavaScript	Falsificación Algorítmica con heurísticas tolerantes (ASI) permitiendo la omisión de cierres funcionales. Posibles bloqueos asíncronos muertos	Flujo dinámico multidimensional escalonado (Motor V8: JIT Ignition/Sparkplug y compiladores paramétricos TurboFan). Clases Ocultas ICs	Infraestructura Pipeline Orinoco GC. Interacción generacional dual de procesos algorítmicos Cheney To-From y sub-hilos Mark-Sweep asíncronos puros
-------------------	--	--	--

Tabla 2. Arquitectura de Analizadores Gramaticales y Gestión de Memoria

4.6. Resultados del Benchmarking (Algoritmo de Collatz)

Para evaluar el impacto de las tecnologías de compilación y ejecución, se diseñó un algoritmo intensivo en cómputo que calcula la longitud de la secuencia de la Conjetura de Collatz para cada número entero desde 1 hasta $N = 2.000.000$. A continuación se presentan los resultados obtenidos en el hardware local:

Lenguaje	Paradigma	Mecanismo	Tiempo	Memoria	Vel. Rel.
Python	Multiparadigma (POO/Imp)	Interpretado (VM)	151557.35 ms	0.0003 MB	1.0x
JavaScript	Multiparadigma (Prot/Func)	JIT (V8 Engine)	2401.99 ms	40.9648 MB	63.1x
Rust	Multiparadigma (Func/Imp)	Nativo (LLVM)	1848.26 ms	1.2500 MB	82.0x
Zig	Imperativo	Nativo (LLVM)	1981.14 ms	1.1000 MB	76.5x

Tabla 3. Comparativa de rendimiento y consumo de memoria por lenguaje (Algoritmo de Collatz)

Análisis del Benchmark: Los resultados evidencian la enorme diferencia entre los lenguajes compilados nativamente (Rust y Zig), que completan la tarea en milisegundos gracias al diseño optimizado de bucles por LLVM, frente a Python, que sufre penalizaciones por la sobrecarga del bucle de evaluación e introspección de tipos en su máquina virtual.

Actividad III: Diseño de un Lenguaje de Dominio Específico (DSL)

5. Actividad III: Diseño de un Lenguaje de Dominio Específico (DSL)

Para solucionar de forma segura el control de la planta industrial crítica ECO-GRID (microrredes y almacenamiento de energía), se presenta el diseño formal del Lenguaje L.

5.1. Especificación del Alfabeto y Reglas Léxicas

- * **Identificadores:** `[a-zA-Z][a-zA-Z0-9]*`
- * **Literales Numéricos:** `[0-9]+` (Enteros representando valores de potencia en kW o temperaturas en °C)
- * **Operador de Asignación:** `:=`
- * **Delimitador de Instrucción:** `;`
- * **Delimitadores de Parámetros:** `( y )`
- * **Comentarios:** Delimitados por `#` al inicio de la línea.
- * **Ignorados:** Espacios en blanco, tabulaciones y saltos de línea (no significativos).

5.2. Palabras Clave Obligatorias

- * **init_grid:** Inicializa el driver de hardware de ECO-GRID.
- * **leer_temperatura(bateria_id):** Retorna la temperatura de la celda de batería en °C.
- * **estado_carga(bateria_id):** Retorna el porcentaje de carga 0-100%.
- * **conmutar_linea(sector_id, estado):** Actúa sobre los relés: 1 = conectar, 0 = aislar.
- * **si_verdadero ... entonces ... fin_si:** Estructura condicional.
- * **mientras ... ejecutar ... fin_mientras:** Estructura repetitiva.

5.3. Gramática Sintáctica Abstracta en EBNF

```

<programa> ::= "init_grid" ";" <sentencia>*
<sentencia> ::= <asignacion> | <condicional> | <bucle> | <llamada_accion> ";"
<asignacion> ::= <identificador> ":=" <expresion> ";"
<condicional> ::= "si_verdadero" <comparacion> "entonces" <sentencia>* "fin_si"
<bucle> ::= "mientras" <comparacion> "ejecutar" <sentencia>* "fin_mientras"
<comparacion> ::= <expresion> <operador_rel> <expresion>
<operador_rel> ::= ">" | "<" | "==" | ">=" | "<=" | "!="
<expresion> ::= <identificador> | <numero> | <llamada_lectura>
<llamada_lectura> ::= "leer_temperatura" "(" <expresion> ")" | "estado_carga" "("
<expresion> ")"
<llamada_accion> ::= "conmutar_linea" "(" <expresion> "," <expresion> ")"
<numero> ::= [0-9]+

```

5.4. Escenario Operativo A: Prevención de Fuga Térmica

Este programa monitoriza la temperatura del banco de baterías 1 de forma continua. Si excede 55°C, se aísla térmicamente apagando la carga solar, activando ventiladores auxiliares y derivando la carga del sector industrial (línea 1) hacia la red comercial de respaldo (línea 2).

```
init_grid;
mientras 1 == 1 ejecutar
    temp := leer_temperatura(1);
    si_verdadero temp > 55 entonces
        conmutar_linea(5, 1); # Activar ventilador de refrigeración auxiliar
(Línea 5)
        conmutar_linea(4, 0); # Desconectar arreglos de Paneles Solares (Línea 4)
para frenar carga
        conmutar_linea(1, 0); # Desconectar el sector industrial de las baterías
(Línea 1)
        conmutar_linea(2, 1); # Conectar el sector industrial a la red de
respaldo comercial (Línea 2)
    fin_si
fin_mientras
```

5.5. Escenario Operativo B: Balance de Carga y Optimización Energética

Este script evalúa el estado de carga de las baterías. Si la carga supera el 90% y hay excedente solar, activa relés para vender electricidad. Si la carga cae por debajo del 20% durante la noche, apaga los sectores de consumo no esenciales para reservar energía en áreas críticas (médicas y servidores).

```
init_grid;
mientras 1 == 1 ejecutar
    carga := estado_carga(1);
    si_verdadero carga > 90 entonces
        # Carga óptima: inyectar energía a la red general pública (Línea 3)
        conmutar_linea(3, 1);
    fin_si

    si_verdadero carga < 20 entonces
        # Batería en nivel crítico: aislar zonas industriales no esenciales (Línea
1)
        # y mantener encendidos servidores y áreas médicas (Línea 6)
        conmutar_linea(1, 0);
        conmutar_linea(6, 1);
    fin_si
fin_mientras
```

6. Conclusiones

1. El Lenguaje como Arquitectura: Los lenguajes de programación no son entes abstractos rígidos; sus reglas morfológicas e intérpretes determinan de forma fundamental el rendimiento del sistema final.

2. Desempeño Comparativo: El análisis de benchmarking evidencia que para sistemas críticos y de alto rendimiento, los lenguajes de compilación nativa (Rust y Zig) son indispensables debido a su mínima latencia y gestión de memoria de forma determinista y predecible.

3. Valor de los DSL: La creación de lenguajes de dominio específico como el Lenguaje L permite a los operadores de industrias críticas interactuar de forma segura con el hardware, limitando el espacio de errores lógicos.

7. Referencias Bibliográfica

Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2008). *Compiladores: Principios, técnicas y herramientas (2da ed.)*. Pearson Educación.

Ecma International. (2025). *ECMAScript 2025 Language Specification*. <https://tc39.es/ecma262/>

Hopcroft, J. E., Motwani, R., & Ullman, J. D. (2008). *Introducción a la teoría de autómatas, lenguajes y computación (3ra ed.)*. Addison-Wesley.

Python Software Foundation. (2026). *The Python Language Reference (v3.12)*. <https://docs.python.org/3/reference/>

Rust Project Developers. (2026). *The Rust Reference*. <https://doc.rust-lang.org/reference/>

Zig Software Foundation. (2026). *Zig Language Reference*. <https://ziglang.org/documentation/>